

K8S-05: Workload Monitoring

Series: K8S — Kubernetes Monitoring | **Notebook:** 5 of 13 | **Created:** January 2026
| **Last Updated:** 08/11/2026

Application-Level Observability in Kubernetes

Workload monitoring focuses on the application layer: deployments, pods, containers, and the services they provide. This notebook covers monitoring Kubernetes workloads from deployment health to service performance.

Table of Contents

- 1. [Workload Types and Monitoring](#)
- 2. [Deployment Health](#)
- 3. [Pod and Container Metrics](#)
- 4. [Service Performance](#)
- 5. [Distributed Tracing in K8s](#)
- 6. [Application Logs](#)
- 7. [Workload Anomalies](#)

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Kubernetes monitoring
DynaKube	cloudNativeFullStack or applicationMonitoring

Requirement	Details
Permissions	<code>metrics.read</code> , <code>entities.read</code> , <code>logs.read</code>
Knowledge	K8S-01 Fundamentals, K8S-04 Cluster Monitoring

1. Workload Types and Monitoring

Kubernetes Workload Controllers

Workload Type	Use Case	Monitoring Focus
Deployment	Stateless apps	Replica availability, rollout status
StatefulSet	Stateful apps (DBs)	Pod identity, persistent storage
DaemonSet	Node-level agents	Coverage across all nodes
Job	Batch processing	Completion rate, duration
CronJob	Scheduled tasks	Execution timing, failures

Dynatrace Workload Entities

Entity Type	Kubernetes Resource	Key Attributes
<code>CLOUD_APPLICATION</code>	Deployment, StatefulSet	Replicas, strategy
<code>CLOUD_APPLICATION_NAMESPACE</code>	Namespace	Labels, quotas
<code>PROCESS_GROUP_INSTANCE</code>	Pod/Container	Resources, image
<code>SERVICE</code>	Service (detected)	Endpoints, traffic

```
// List Kubernetes workloads (Deployments) via smartscape topology
smartscapeNodes "K8S_DEPLOYMENT"
| fields entity.name = name, tags
| sort entity.name asc
| limit 50

// For StatefulSets, DaemonSets, etc., substitute the type:
// smartscapeNodes "K8S_STATEFULSET"
// smartscapeNodes "K8S_DAEMONSET"

// Legacy alternative (deprecated for new content):
// fetch dt.entity.cloud_application
// | fields entity.name, tags
// | sort entity.name asc
// | limit 50
```

2. Deployment Health

Deployment Status Indicators

Metric	Description	Healthy State
Available Replicas	Pods passing readiness	Equals desired
Ready Replicas	Pods passing all probes	Equals desired
Updated Replicas	Pods with latest spec	Equals desired
Collision Count	Hash collisions	Zero

Rollout Monitoring

During deployments, monitor:

- Old ReplicaSet scaling down
- New ReplicaSet scaling up
- Pod readiness during transition

```
// Deployment events - track rollouts
fetch logs, from:-1h
| filter matchesPhrase(content, "deployment") and (matchesPhrase(content,
"scaled") or matchesPhrase(content, "updated") or matchesPhrase(content,
"rollout"))
| fields timestamp, content
| sort timestamp desc
| limit 30
```

```
// Pod restart counts - identify unstable workloads
fetch logs, from:-1h
| filter matchesPhrase(content, "restarted") or matchesPhrase(content,
"BackOff")
| fields timestamp, content
| sort timestamp desc
| limit 30
```

3. Pod and Container Metrics

Container Resource Metrics

Metric	Description	Alert Threshold
CPU Usage %	Usage vs. limit	>85% sustained
Memory Usage %	Usage vs. limit	>90%
CPU Throttled	Time throttled	>10% of time
Memory Working Set	Active memory	Approaching limit

Pod Lifecycle States

State	Description	Action
Pending	Waiting to schedule	Check resources, node selectors
Running	At least one container running	Normal
Succeeded	All containers exited 0	Job completed
Failed	Containers exited non-zero	Check logs

State	Description	Action
Unknown	Node communication lost	Check node health

```
// Container CPU usage - find high consumers
timeseries avgCpuUsageMillicores = avg(dt.kubernetes.container.cpu_usage),
from:-1h, by:{dt.entity.container_group_instance}
| sort avgCpuUsageMillicores desc
| limit 15
```

```
// Container memory usage approaching limits
timeseries avgMemUsageBytes = avg(dt.kubernetes.container.memory_working_set),
from:-1h, by:{dt.entity.container_group_instance}
| fieldsAdd avgMemUsageBytesValue = arrayAvg(avgMemUsageBytes)
| sort avgMemUsageBytesValue desc
| limit 15
```

```
// Container CPU throttling - performance impact
timeseries avgThrottled = avg(dt.containers.cpu.throttled_time), from:-1h, by:
{dt.entity.container_group_instance}
| fieldsAdd avgThrottledValue = arrayAvg(avgThrottled)
| filter avgThrottledValue > 0
| sort avgThrottledValue desc
| limit 15
```

4. Service Performance

Service-Level Metrics

Metric	Description	SLO Example
Response Time	P50, P90, P99 latency	P99 < 500ms
Error Rate	Failed requests %	< 0.1%
Throughput	Requests per second	Based on capacity
Availability	Successful responses	> 99.9%

Golden Signals for Services

Signal	What to Monitor	DQL Approach
Latency	Response time distribution	Timeseries with percentiles
Traffic	Request rate	Count over time
Errors	Error rate, types	<code>countIf(span.status_code == "error")</code>
Saturation	Resource utilization	CPU, memory, connections

Span failure status: three traps, all silent. Getting error rate right on spans depends on three facts, and each of them fails by returning a plausible number rather than an error.

Trap	What happens
<code>otel.status_code</code>	The field does not exist — it has no row in <code>dt.semantic_dictionary.fields</code> . The real field is <code>span.status_code</code> (<code>stable</code>). Likewise <code>otel.status_message</code> → <code>span.status_message</code> (<code>experimental</code> , and frequently null even on failing spans).
<code>== "ERROR"</code>	Values are lowercase . On a live tenant <code>span.status_code == "ERROR"</code> matched 0 spans against 18,630 for <code>"error"</code> in the same two-hour window (08/11/2026). A rename without the case fix converts a hard failure into a silent zero.
<code>countIf(span.status_code != "error")</code>	<code>span.status_code</code> is null on virtually every successful span — <code>"ok"</code> is written so rarely it is noise (9 spans out of ~659,000 in that window). A <code>!=</code> comparison against null yields null, not true, so this counts almost nothing. Derive successes as total - errors .

The third trap is the expensive one: an availability SLO built on `countIf(... != "error")` reads **0% available** on a healthy service, and nothing in the query errors to tell you.

```
// Service response time (spans)
fetch spans, from:-1h
| filter span.kind == "server"
| summarize
    p50 = percentile(duration, 50),
    p90 = percentile(duration, 90),
    p99 = percentile(duration, 99),
    by:{dt.entity.service}
| sort p99 desc
| limit 15
```

```
// Service error rates
// The field is span.status_code (stable). otel.status_code does not exist.
// Values are lowercase ("error"), and the field is NULL on successful spans –
// so successes must be derived as total - errors, never counted directly.
fetch spans, from:-1h
| filter span.kind == "server"
| summarize {
    total = count(),
    errors = countIf(span.status_code == "error")
}, by:{dt.entity.service}
| fieldsAdd successes = total - errors
| fieldsAdd errorRate = round(100.0 * errors / total, decimals: 2)
| filter errors > 0
| sort errorRate desc
| limit 15
```

```
// Service throughput
fetch spans, from:-1h
| filter span.kind == "server"
| summarize requestCount = count(), by:{dt.entity.service}
| sort requestCount desc
| limit 10
```

5. Distributed Tracing in K8s

Trace Context in Kubernetes

Distributed tracing in Kubernetes follows requests across:

- Ingress → Services

- Service → Service (east-west traffic)
- Service → External (databases, APIs)

Trace Attributes in K8s

Attribute	Source	Use Case
<code>k8s.namespace.name</code>	OneAgent	Filter by namespace
<code>k8s.deployment.name</code>	OneAgent	Identify workload
<code>k8s.pod.name</code>	OneAgent	Instance-level analysis
<code>k8s.container.name</code>	OneAgent	Container identification

```
// Traces by namespace
fetch spans, from:-1h
| filter isNotNull(k8s.namespace.name)
| summarize requestCount = count(), avgDuration = avg(duration), by:
{k8s.namespace.name}
| sort requestCount desc
| limit 15
```

```
// Slow traces by workload
fetch spans, from:-1h
| filter span.kind == "server" and duration > 1000000000 // > 1 second
| fields timestamp, trace.id, k8s.namespace.name, k8s.deployment.name,
duration
| sort duration desc
| limit 20
```

6. Application Logs

Container Log Collection

OneAgent collects container logs from:

- stdout/stderr streams
- Mounted log files (with configuration)

Log Attributes

Attribute	Description
k8s.namespace.name	Source namespace
k8s.pod.name	Source pod
k8s.container.name	Source container
dt.entity.container_group_instance	Entity relationship

```
// Application error logs by namespace
fetch logs, from:-1h
| filter loglevel == "ERROR" or loglevel == "SEVERE"
| filter isNotNull(k8s.namespace.name)
| summarize errorCount = count(), by:{k8s.namespace.name}
| sort errorCount desc
| limit 15
```

```
// Recent application logs with context
fetch logs, from:-1h
| filter isNotNull(k8s.namespace.name)
| filter loglevel == "ERROR"
| fields timestamp, k8s.namespace.name, k8s.pod.name, content
| sort timestamp desc
| limit 30
```

7. Workload Anomalies

Common Anomaly Types

Anomaly	Indicators	Root Cause
Memory Leak	Steadily increasing memory	Application bug
CPU Spike	Sudden CPU increase	Traffic surge, infinite loop
Error Storm	Rapid error increase	Dependency failure
Latency Degradation	Increasing response time	Resource contention

Dynatrace Intelligence for Workloads

Dynatrace Dynatrace Intelligence automatically detects:

- Response time degradation
- Error rate increases
- Resource saturation
- Failure rate anomalies

```
// Error count by namespace (detecting increases)
fetch logs, from:-1h
| filter loglevel == "ERROR"
| filter isNotNull(k8s.namespace.name)
| summarize errorCount = count(), by:{k8s.namespace.name}
| sort errorCount desc
| limit 10
```

```
// Memory usage by workload (detecting high consumers)
timeseries avgMemoryBytes = avg(dt.kubernetes.container.memory_working_set),
from:-1h, by:{dt.entity.cloud_application}
| sort avgMemoryBytes desc
| limit 10
```

Next Steps

With workload monitoring configured, proceed to:

Next Notebook	Topic
K8S-06: Namespace Organization	Boundaries and access control
K8S-07: Events and Logs	Kubernetes event analysis
K8S-08: DQL for Kubernetes	Advanced query patterns

Summary

In this notebook, you learned:

- Kubernetes workload types and their monitoring focus
 - Deployment health indicators and rollout monitoring
 - Pod and container resource metrics
 - Service-level performance monitoring (golden signals)
 - Distributed tracing with Kubernetes attributes
 - Application log analysis by namespace and pod
 - Workload anomaly detection patterns
-

References

- [Set up Dynatrace on Kubernetes \(DT docs\)](#)
 - [Full observability deployment \(DT docs\)](#)
 - [Application observability deployment \(DT docs\)](#)
 - [Kubernetes app — workloads view \(DT docs\)](#)
 - [Container platform monitoring \(DT docs\)](#)
 - [Service monitoring \(DT docs\)](#)
 - [smartscapeNodes command \(DT docs\)](#)
-

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.